

Matplotlib ve Seaborn'a Giriş

Veri Madenciliği - Hafta 6

Veri Görselleştirme: Matplotlib ve Seaborn'a Giriş

Veri analizinin kritik aşamalarından biri olan görselleştirme konusuna bu hafta odaklanılacaktır.

Matplotlib ve Seaborn kütüphanelerinin temel işlevleri ve veri setlerindeki kalıpları keşfetme yeteneği kazanımı hedeflenmektedir.

Sunum, temel çizim türlerinden (çizgi, çubuk, histogram) istatistiksel grafiklere (Seaborn) kadar uzanmaktadır.



Veri Görselleştirmenin Tanımı ve Önemi

Veri görselleştirme, verilerdeki temel anlamı iletmek için grafikler ve çizimler oluşturma işlemidir.

Bu süreç, büyük ve karmaşık veri setlerinin insan zihni tarafından daha hızlı işlenmesini sağlar.

Nicel verilerin kolay anlaşılır görsel temsillerine dönüştürülmesidir (Örnek: Zaman serileri, coğrafi veriler).



Görselleştirmenin Kritik Amaçları

Verileri daha iyi anlamaya yardımcı olmak.
Aykırı değerleri (Outliers) ve anomalileri belirlemek için grafikler oluşturmak.
Eksik verilerin (Missing Data) dağılımını veya desenlerini tespit etmek.
Değişkenler arasındaki potansiyel ilişkileri veya korelasyonları keşfetmek.

Python Ekosisteminde Görselleştirme Araçları

Pandas, veri manipülasyonu için temel yapıyı sağlar.

****Matplotlib:**** Python'un en eski ve en esnek çizim kütüphanesidir. Temel düzeyde ve tam kontrol gerektiren çizimler için kullanılır.

****Seaborn:**** Matplotlib üzerine inşa edilmiştir. Daha estetik, modern ve istatistiksel grafikler sunmak üzere tasarlanmıştır.

Diğerleri: Plotly (etkileşimli), Bokeh (web tabanlı), Altair (deklaratif).

Matplotlib Kütüphanesine Giriş

Matplotlib, 2D çizimler için kapsamlı bir kütüphanedir. Piyasada bulunan bilimsel ve mühendislik grafiklerinin çoğunu oluşturmak için kullanılabilir. Çoğunlukla 'pyplot' modülü altında kullanılır (plt takma adı ile içe aktarılır).



Matplotlib Mimarisi: Figure, Axes ve Axis

****Figure (Şekil/Tuval):**** Tüm çizimin yer aldığı en dıştaki kapsayıcıdır. Pencerenin tamamını temsil eder.

****Axes (Eksenler/Plot):**** Grafiğin kendisinin çizildiği alandır. Bir Figure birden çok Axes içerebilir (subplot mantığı).

****Axis (Eksen):**** Sayısal aralıkları belirleyen x ve y eksenleridir. Etiketleri, başlıkları ve tik işaretlerini içerir.



Temel Kurulum ve İçe Aktarma

Matplotlib'i kullanmak için standart komut: `pip install matplotlib`
Genellikle NumPy (sayısal işlemler) ile birlikte kullanılır.
Standart içe aktarma biçimi:
`import matplotlib.pyplot as plt`
`import numpy as np`



Çizgi Grafik (Line Plot): Kullanım Alanları

En yaygın kullanılan grafik türüdür.

Zaman içindeki trendleri veya sürekli bir değişkenin değerine göre değişimini göstermek için idealdir.

Örnekler: Hisse senedi fiyatlarındaki değişim, sıcaklık artışı, bir denklem fonksiyonunun grafiği.



Çizgi Grafik Oluşturma Söz Dizimi

`plt.plot(x, y)` komutu kullanılır.

`x` ve `y`, genellikle NumPy dizileri (array) veya listelerdir.

Grafiğin gösterilmesi için `plt.show()` komutu zorunludur.

Örnek koddan: `x = np.arange(25)`

`y = x**2 + 1`

`plt.plot(x,y)`

Grafik Başlıkları ve Eksen Etiketleri

Grafiği tanımlayan bir başlık eklenmelidir: `plt.title('Denklem Grafiği')`.
X ekseninin neyi temsil ettiğini belirtmek: `plt.xlabel('X Eksen')`.
Y ekseninin neyi temsil ettiğini belirtmek: `plt.ylabel('Y Eksen')`.
Bu meta veriler, görselin tek başına anlaşılabilir olmasını sağlar.

Çoklu Çizgiler ve Lejant (Gösterge)

Aynı Axes üzerinde birden fazla `plt.plot()` komutu çalıştırılarak çoklu çizgiler çizilebilir.
Her bir çizgiye bir etiket (label) verilmelidir.
Grafik içindeki çizgilerin hangi veriye ait olduğunu açıklamak için `plt.legend()` kullanılır.



Çizgi Stilleri ve İşaretleyiciler (Markers)

Çizgi kalınlığı, rengi ve stili ayarlanabilir (Örn: kesik çizgi, noktalı çizgi).
Veri noktalarını vurgulamak için işaretleyiciler (o, s, ^ gibi semboller) kullanılabilir.

Biçimlendirme, 'plt.plot(x, y, 'r--o')' gibi kısa formatlarla veya keyword argümanlarla yapılabilir.



Eksen Aralıklarını Ayarlama ve Kısıtlama

Bazen tüm veri aralığı yerine belirli bir bölgeye odaklanmak gerekebilir.
X eksenini sınırlarını belirlemek: `plt.xlim(min_deger, max_deger)`.
Y eksenini sınırlarını belirlemek: `plt.ylim(min_deger, max_deger)`.

Izgara Çizgileri (Grids) ve Notasyonlar

Grafiğin arkasına ızgara çizgileri eklemek, veri noktalarının değerlerini okumayı kolaylaştırır: `plt.grid(True)`.

Belirli noktaları veya bölgeleri vurgulamak için oklar (arrow) ve metin notları (`plt.annotate`) kullanılabilir.

Çubuk Grafik (Bar Graph): Kategorik Karşılaştırma

Çubuk grafikler, farklı kategoriler arasındaki verileri karşılaştırmak için kullanılır. Genellikle kategorik bir değişken (x eksen) ve ona karşılık gelen sayısal bir değer (y eksen) içerir.

Örnekler: Satış miktarları, oylama sonuçları, programlama dillerinin popülaritesi.



Çubuk Grafik Oluşturma Söz Dizimi

`plt.bar(kategoriler, değerler)` komutu kullanılır.
Kategoriler genellikle dizilerden (string) oluşur (Örn: 'C', 'Python').
Değerler sayısal olmalıdır (Örn: 88, 92).
Matplotlib kod örneği: `plt.bar(diller, oranlar)`.

Yatay Çubuk Grafikler (Barh)

Dikey çubuk (plt.bar) yerine yatay çubuklar (plt.barh) kullanılabilir. Yatay grafikler, özellikle kategori isimleri uzun olduğunda okunabilirliği artırır. X eksenini değerleri temsil ederken, Y eksenini kategorileri temsil eder.

Gruplanmış Çubuk Grafikler (Grouped Bar Charts)

Farklı grupların alt kategorilerini yan yana karşılaştırmak için kullanılır.
Örneğin: 2023 ve 2024 yıllarındaki satışları bölge bazında karşılaştırmak.
Matplotlib'de çubukların konumları (x değerleri) ve genişlikleri manuel olarak ayarlanmalıdır.

Yığılmış Çubuk Grafikler (Stacked Bar Charts)

Bir kategorinin toplam değerinin, alt bileşenler tarafından nasıl oluşturulduğunu gösterir.

'bottom' parametresi kullanılarak bir önceki serinin üzerine yığılma sağlanır. Veri setindeki toplam payları veya katkıları analiz etmek için kullanışlıdır.



Histogram Grafik: Dağılımın Görselleştirilmesi

Histogramlar, bir dağılımı göstermek için kullanılır. Sürekli sayısal verilerin belli aralıklara (bin'lere) bölünerek, her aralığa düşen veri noktası sayısını (frekansını) gösterir.



Histogram Söz Dizimi ve Bin Kavramı

`plt.hist(veri, bins=sayı)` komutu kullanılır.

****Bin (Kutu/Aralık):**** Veri aralığının kaç eşit parçaya bölüneceğini belirler. Bin sayısı azsa detay kaybolur; çoksa grafik gürültülü hale gelebilir.

Örnek kod: `data = np.random.randint(0,100,100)`
`plt.hist(data)`

Histogram Tipleri ve Normalizasyon

Frekans (count) yerine yoğunluğu (density) göstermek, farklı büyüklükteki veri setlerinin dağılımlarını karşılaştırmayı kolaylaştırır.

'density=True' parametresi kullanılır.

Kümülativ histogramlar (cumulative=True) da dağılımın kümülatif toplamını gösterir.

Dağılım Grafik (Scatter Plot): İlişki Analizi

Dağılım grafikleri, iki sayısal değişkeni karşılaştırmak için kullanılır. Her nokta, veri setindeki bir gözlemi temsil eder. Temel amaç, değişkenler arasında bir korelasyon (ilişki) olup olmadığını görsel olarak tespit etmektir.

Dağılım Grafik Oluşturma Söz Dizimi

`plt.scatter(x_veri, y_veri)` komutu kullanılır.
Matplotlib, her (x, y) çifti için tuval üzerine bir nokta çizer.
Veri setindeki değişkenlerin doğrusallık, kümelenme veya aykırı değer yapıları bu grafikte netleşir.

Üçüncü Boyutun Kodlanması

Dağılım grafikleri, nokta büyüklüğünü (s - size) ve rengini (c - color) üçüncü ve dördüncü bir değişkeni temsil etmek için kullanabilir.
Bu sayede, iki boyutlu bir düzlemde çok değişkenli analiz yapılabilir.
Örnek: X=Gelir, Y=Yaş olsun. Nokta büyüklüğü=Harcama miktarını gösterebilir.



Korelasyonun Görsel Olarak Yorumlanması

Noktalar yukarı doğru bir eğilim gösteriyorsa: Pozitif Korelasyon.

Noktalar aşağı doğru bir eğilim gösteriyorsa: Negatif Korelasyon.

Noktalar rastgele dağılmışsa: Korelasyon yok.

Dağılım grafiği, regresyon analizi öncesinde veri yapısını anlamak için ilk adımdır.



Pasta Grafik (Pie Chart): Oransal Temsil

Pasta grafik, her dilimin bir kategoriye temsil ettiđi grafik türüdür. Genellikle bütünün yüzdesel dağılımını veya oransal katkısını göstermek için kullanılır. Dilimlerin toplamı %100'e (veya 360 dereceye) eşit olmalıdır.

Pasta Grafik Oluşturma Söz Dizimi

`plt.pie(değerler, labels=kategoriler)` komutu kullanılır.
'autopct' parametresi, her dilimin yüzdesini otomatik olarak hesaplayıp üzerine yazdırır (Örn: `autopct='%1.1f%%'`).
Dilimlerin ayrılması (`explode`) gibi özelleştirmeler yapılabilir.

Pasta Grafiğın Kullanım Sınırlamaları

Çok fazla kategori varsa (7'den fazla), okunabilirlik hızla düşer.
İnsan gözü, dilim alanlarını karşılaştırmada zorlanır.

****Alternatif:**** Oransal karşılaştırma için genellikle sıralı Çubuk Grafikler (Bar Charts) veya Yığılmış Çubuk Grafikler önerilir.



Kutu Grafiđi (Box Plot) Kavramı

Kutu grafiđi, verinin dađılımını, medyanını ve aykırı deđerlerini beř temel sayısal özet (five-number summary) üzerinden gösterir. Veri setindeki çarpıklıđı, simetriyi ve yayılımı anlamak için çok kullanışlıdır. Temel olarak, merkez eğilimi ve deđişkenliđi görselleřtirir.

Kutu Grafiğinin Beş Sayısal Özeti

1. Minimum: En küçük gözlem değeri (aykırı değer hariç).
2. Q1 (Birinci Çeyreklik): Verinin %25'i bu noktanın altındadır.
3. Medyan (Q2): Verinin %50'si (ortanca değer).
4. Q3 (Üçüncü Çeyreklik): Verinin %75'i bu noktanın altındadır.
5. Maksimum: En büyük gözlem değeri (aykırı değer hariç).

Kutu Grafiği ve Aykırı Değer Tespiti

Kutu uzunluğu, Çeyrekler Arası Aralık ($IQR = Q3 - Q1$) olarak bilinir.
Aykırı değerler, genellikle $Q3 + 1.5 * IQR$ veya $Q1 - 1.5 * IQR$ dışında kalan noktalar olarak tanımlanır.
Matplotlib/Seaborn bu aykırı değerleri nokta olarak gösterir.

Matplotlib Subplots: Çoklu Grafikler

Aynı tuvalde birden fazla grafik yan yana veya alt alta çizmek için kullanılır. 'plt.subplot(nrows, ncols, index)' veya daha modern 'plt.subplots()' fonksiyonu tercih edilir.

Bu, farklı değişkenler arasındaki ilişkileri veya aynı değişkenin farklı alt gruptaki dağılımını karşılaştırmayı sağlar.



Matplotlib Nesne Yönelimli Yaklaşım (OO)

Matplotlib'in iki arayüzü vardır: Pyplot (hızlı çizim) ve Nesne Yönelimli (tam kontrol).

Üniversite düzeyinde karmaşık görselleştirmeler için OO yaklaşımı önerilir. OO: 'fig, ax = plt.subplots()' ile Figure ve Axes nesneleri oluşturulur, çizimler 'ax.plot()' veya 'ax.bar()' ile yapılır.

Matplotlib Stil Yönetimi (Stylesheets)

Varsayılan Matplotlib görünümü bazen basit kalabilir. Farklı stil sayfaları (styles) kullanarak grafiklerin genel görünümünü hızla değiştirebiliriz (Örn: 'ggplot', 'dark_background', 'seaborn'). 'plt.style.use('stil_adi')' ile uygulanır.



Seaborn Kütüphanesine Geçiş

Seaborn, Matplotlib üzerine inşa edilmiş yüksek seviyeli bir kütüphanedir. Pandas DataFrame yapılarıyla doğrudan çalışmak üzere tasarlanmıştır, bu da veri analizi kodunu basitleştirir. Temel odak noktası, karmaşık istatistiksel ilişkileri gösteren estetik ve bilgilendirici grafikler oluşturmaktır.

Seaborn'un Matplotlib'e Göre Avantajları

Daha yüksek düzeyde API: Tek bir fonksiyon çağırısı ile karmaşık istatistiksel çizimler (Örn: regresyon çizgisi) oluşturabilir.
Varsayılan olarak daha çekici renk paletleri ve stiller sunar.
Categorical, distribution, ve relational plot'lar için özelleşmiş modülleri vardır.
Birden çok değişkeni tek bir görselde birleştirme yeteneği.

Seaborn Kurulum ve İçe Aktarma

Kurulum: `pip install seaborn`

Genellikle 'sns' takma adıyla içe aktarılır:

```
import seaborn as sns
```

Seaborn, çoğu zaman Matplotlib'in temel ayarlarını otomatik olarak değiştirir (arka plan, renk paletleri vb.).

Seaborn Dağıtım Grafikleri (Distribution Plots)

- `sns.histplot()`**: Histogram ve isteğe bağlı yoğunluk tahmini.
- `sns.kdeplot()`**: Çekirdek Yoğunluk Tahmini (Kernel Density Estimate) ile verinin pürüzsüzleştirilmiş dağılımını gösterir.
- `sns.rugplot()`**: Her bir veri noktasını eksen üzerinde küçük bir çizgiyle işaretler.

İki Değişkenli İlişki Grafikleri (Relational Plots)

****sns.scatterplot():**** Matplotlib'deki dağılım grafiğinin Seaborn karşılığıdır, ancak estetik ayarları ve alt gruplama (hue) desteği daha gelişmiştir.

****sns.lineplot():**** Özellikle zaman serileri için uygun, gelişmiş çizgi grafikleri sağlar.

Bu fonksiyonlar, bir veri kümesinin içindeki ilişkilerin dağılımını ve yapısını anlamaya odaklanır.



Kategoriye Dayalı Grafikler (Categorical Plots)

Kategorik bir değişken ile sayısal bir değişken arasındaki ilişkiyi gösterirler.

- `sns.barplot()`**: Matplotlib çubuk grafiğine benzer, ancak otomatik olarak ortalamayı ve güven aralığını hesaplar.
- `sns.boxplot()`**: Matplotlib kutu grafiğinin daha estetik versiyonu.
- `sns.violinplot()`**: Kutu grafiği ile KDE'yi birleştirerek dağılımın tam şeklini gösterir.

İstatistiksel Tahmin Grafikleri (Estimates)

Seaborn'daki 'barplot' ve 'pointplot' gibi grafikler, varsayılan olarak ortalamayı ve standart sapma veya güven aralığını (confidence interval) çizer. Bu, sadece nokta tahmini değil, tahmini ne kadar güvenilir olduğunu da görselleştirmeyi sağlar.

Korelasyon Grafikleri: Isı Haritaları (Heatmaps)

`**sns.heatmap():**` Veri matrisindeki veya korelasyon matrisindeki değerleri renk yoğunluğu ile temsil eder.

Genellikle, bir veri setindeki tüm değişken çiftleri arasındaki korelasyon katsayılarını göstermek için kullanılır.

Renk skalası (cmap), değerlerin büyüklüğünü hızlıca anlamayı sağlar.

Regresyon Grafikleri (Regression Plots)

****sns.lmplot() ve sns.regplot():**** Dağılım grafiği üzerine otomatik olarak doğrusal bir regresyon çizgisi (en küçük kareler yöntemi) ve güven aralığı çizer.

Bu, iki değişken arasındaki doğrusal trendi hızlıca değerlendirmeye yardımcı olur.

Alt gruplama (hue) ile farklı gruplar için ayrı regresyon çizgileri çizilebilir.

Pair Plot (Çift Grafik): Veri Setine Hızlı Bakış

****sns.pairplot():**** Veri setindeki tüm sayısal değişken çiftleri arasındaki dağılım grafiklerini (scatter plot) ve her değişkenin tekli dağılımını (histogram/kde) gösteren bir matris oluşturur. Veri madenciliğine başlamadan önce ilişkileri ve dağılımları hızlıca keşfetmek için paha biçilmezdir.

Matplotlib ve Seaborn'un Birlikte Kullanımı

Seaborn, temel çizim için Matplotlib'in Figure ve Axes yapısını kullanır. Seaborn ile grafiği çizin (kolay istatistiksel çizim). Ardından, Matplotlib'in Nesne Yönelimli arayüzünü kullanarak (`ax.set_title()`, `ax.tick_params()`) grafiğe ince ayar yapın.

Renk Paletleri ve Estetik Yönetimi

Seaborn, görsel olarak optimize edilmiş yerleşik renk paletlerine sahiptir (sequential, diverging, categorical).

Renk seçimi, verinin türüne (sıralı, kategorik, nicel) uygun olmalıdır.

Renk Körlüğü Dostu Paletler (Colorblind-friendly palettes) kullanımı akademik çalışmalarda etik bir gerekliliktir.



Kategorik Verilerde Sıralama (Ordering)

Kategorik grafiklerde (bar, box, violin), kategorilerin sıralaması grafiğin yorumlanmasını büyük ölçüde etkiler.

Alfabetik sıralama yerine, değer büyüklüğüne göre sıralama yapmak (en büyükten en küçüğe), karşılaştırmayı kolaylaştırır.

Seaborn'da 'order' parametresi bu kontrolü sağlar.



Grafik Türü Seçim Kriterleri

Hangi değişkenleri inceliyorum (Sayısal? Kategorik?):

Dağılım gösterme: Histogram, KDE, Kutu Grafiği.

Karşılaştırma (Kategorik): Çubuk Grafik.

İlişki gösterme (Sayısal-Sayısal): Dağılım Grafik, Çizgi Grafik (zaman serisi).

Veri Görselleřtirmenin Temel İlkeleri

Edward Tufte'nin ilkeleri: Grafiğın mürekkep yoğunluğunu artır (Data-Ink Ratio).
Grafik, hikayeyi anlatmalı, süslemeyi değıl (Chart Junk'tan kaçın).
Veri-bağılam oranını maksimize et (bilgilendirici yoğunluk).



Görselleřtirmede Etik Sorunlar

Eksenlerin sıfırdan başlatılmaması (özellikle çubuk grafiklerde) veriyi manipüle edebilir.

Gereksiz 3D efektleri kullanmak, oransal algıyı bozabilir.

Yanlış renk paletleri veya ölçeklendirmelerle yanıltıcı sonuçlar sunmak.



Uygulama: Matplotlib ile Çizgi Grafik (Code Walkthrough)

```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(25)
y = x**2 + 1
plt.plot(x,y)
plt.title('Karesel Denklem Grafiği')
plt.xlabel('X Değerleri')
plt.ylabel('Y Değerleri')
plt.show()
```

Uygulama: Matplotlib ile Çubuk Grafik (Code Walkthrough)

```
diller = ['C', 'C++', 'Python', 'Java']  
oranlar = (Örnek değerler)  
plt.bar(diller, oranlar)  
plt.title('Dillerin Kullanım Oranları')  
plt.show()
```

Uygulama: Matplotlib ile Histogram (Code Walkthrough)

```
data = np.random.randint(0, 100, 100) (0-100 arasında 100 rastgele tam sayı)  
plt.hist(data, bins=10)  
plt.title('Rastgele Sayıların Dağılımı')  
plt.xlabel('Değer Aralıkları')  
plt.ylabel('Frekans')  
plt.show()
```

Etkileşimli Görselleştirme Araçlarına Kısa Bakış

Matplotlib ve Seaborn statik çıktılar üretirken, modern analizler etkileşim (zoom, hover) gerektirebilir.

****Plotly:**** Çevrimdışı ve çevrimiçi kullanılabilen, zengin etkileşimli grafikler sunar.

****Bokeh:**** Python'da etkileşimli görselleştirmeler oluşturarak web tarayıcılarında yayımlanmasını sağlar.

Bu araçlar, Veri Bilimi projelerinin sunum aşamasında popülerdir.



Büyük Veri Görselleştirmesi Zorlukları

Milyonlarca veri noktasını tek bir grafikte göstermek (overplotting) karmaşaya yol açar.

Çözümler: Veri kümeleme (binning), örnekleme (sampling) veya yoğunluk haritaları (density maps) kullanmak.

Verinin bir özetini görselleştirmek, tüm noktaları çizmekten daha bilgilendiricidir.



Matplotlib'de 3D Görselleştirme

Matplotlib, 3 boyutlu yüzey (surface) veya dağılım (scatter) grafikleri oluşturmak için de kullanılabilir.

Özellikle üç sayısal değişken arasındaki ilişkileri incelemek veya matematiksel yüzeyleri modellemek için uygundur.

Kütüphane: `'from mpl_toolkits import mplot3d'`.



Coğrafi Veri Görselleştirme (Geospatial)

Matplotlib, basemap veya daha modern Cartopy kütüphaneleri ile coğrafi verileri haritalar üzerinde görselleştirmeye olanak tanır.
Örnekler: Nüfus yoğunluğu haritaları, deprem bölgelerinin dağılımı, trafik akışı.



Veri Görselleştirme Sürecinde Hata Ayıklama

Hata: plt.show() unutulması (Grafiğin görüntülenmemesi).

Hata: X ve Y verilerinin boyutlarının eşleşmemesi.

Hata: Kategorik veriyi sayısal değişken olarak işlemeye çalışmak.

Çözüm: Hata mesajlarını dikkatle okumak ve şekil (shape) boyutlarını kontrol etmek.



Görselleştirme Becerilerinin Önemi

Öğrenciler, bu beceriler sayesinde veri setlerindeki ilişkileri, dağılımları ve kalıpları görsel olarak keşfetme ve sunma yeteneği kazanırlar. İyi görselleştirme, karmaşık makine öğrenmesi algoritmalarının sonuçlarının açıklanmasını (Explainable AI) kolaylaştırır. Analitik raporların temelini oluşturur.



Özet: Matplotlib vs Seaborn

Matplotlib: Düşük seviye kontrol, özelleştirilmiş bilimsel çizimler, temel mimari (Figure, Axes) kontrolü gerektiğinde.

Seaborn: Yüksek seviye, estetik çizimler, hızlı istatistiksel analizler, Pandas DataFrame entegrasyonu gerektiğinde.

İdeal Yaklaşım: Seaborn ile hızlı istatistiksel keşif, Matplotlib OO ile son rötuşlar.

Öğrenilen Temel Grafik Türlerinin Kapsamı

Temel Çizimler: Çizgi, Çubuk, Histogram, Dağılım, Pasta Grafikleri.
İstatistiksel Keşif: Kutu Grafiği, KDE, Isı Haritası (Seaborn).
Bu grafik türleri, veri analizinde keşifsel veri analizi (EDA) aşamasının %80'ini oluşturur.



İleri Okuma ve Kaynaklar

Matplotlib Resmi Dokümantasyonu.

Seaborn Resmi Galerisi (çeşitli istatistiksel grafik örnekleri).

Edward Tufte'nin 'The Visual Display of Quantitative Information' adlı klasik eseri.



Soru & Cevap

Bu bölümde, Matplotlib ve Seaborn kütüphaneleriyle ilgili sorularınız yanıtlanacak ve haftanın konuları özetlenecektir.

Matplotlib ve Seaborn'a Giriş

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

